

Programmation Backend

PHP et Programmation Orientée Objet



Pr. ELALAOUI Hasna

h.elalaoui@uca.ac.ma

1

Introduction



Rappel sur la POO

- La programmation orientée objet est plus naturelle donc plus intuitive. Si c'est le cas, c'est parce qu'elle utilise des entités appelées **objets**.
- Un **objet** possède sa propre structure interne qui définit ses propriétés (**attributs**) et son comportement (**méthodes**).
- Si on compare avec le monde réel, les objets sont partout autour de nous. Une personne est un objet, une voiture en est un autre, une maison, une école, un animal de compagnie... tous sont des objets.

Exemple

- Prenons l'objet "voiture". Bien qu'il en existe plusieurs modèles, le fait d'évoquer le mot "voiture" fait penser à des points comme: couleur, options, vitesse, puissance du moteur...
- Cependant, une voiture peut aussi entamer des actions, par exemple: Accélérer, Ralentir, Tourner à droite, Tourner à gauche, Reculer...



Notion de classe

- Les objets de la POO doivent être créés d'abord pour pouvoir être manipulés après. C'est la **classe** qui se charge de donner vie aux objets.
- C'est la classe qui contient la définition des objets qui vont être créés après. En général on considère une classe comme un moule à objets. Avec un seul moule on peut créer autant d'objets que l'on souhaite.
- Techniquement parlant, une classe est une structure qui contient des attributs (appelés aussi **variables membres**) et des méthodes (connus également par **fonctions membres**).
- Les méthodes agissent sur les attributs de cette structure. Par exemple, le fait d'accélérer augmente la vitesse de la voiture.



Principe d'encapsulation

- Le fait d'exposer les attributs aux utilisateurs peut compromettre le bon fonctionnement de la classe. Il faut donc les masquer et leur limiter l'accès uniquement de l'intérieur de la classe par le biais des méthodes prévues à cet effet.
- L'encapsulation est un principe fondamental de la POO. Il vise à masquer les attributs aux utilisateurs du code (les programmeurs qui se serviront de la classe par la suite).
- En fait, ce qui est important dans une classe ce sont les attributs. Les méthodes ne font qu'agir sur ceux ci.

Exemple: Accélérer en conduisant une voiture

- Nous ne manipulons pas les pièces mécaniques (attributs) directement mais en utilisant des pédales (méthodes membres) pour le faire

2

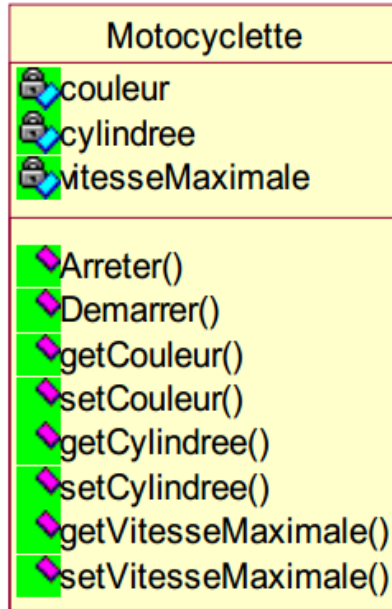
Classe et Objet



Déclaration d'une classe



En PHP5, la définition, la spécification et la réalisation d'une classe sont déclarées dans le même bloc :



```
class Motocyclette{
```

```
//attributs
```

```
private $couleur;  
private $cylindree;  
private $vitesseMaximale;
```

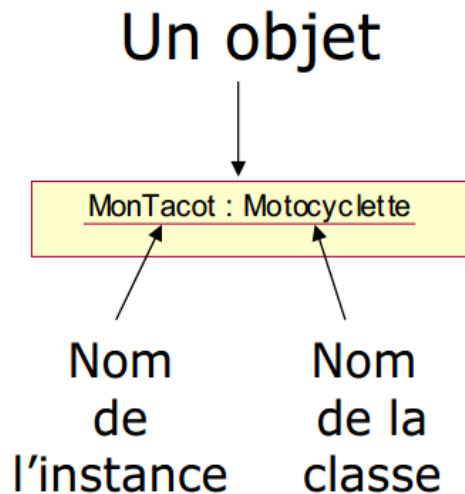
```
//méthodes
```

```
public function getCouleur(){  
    return $this->couleur;  
}  
public function setCouleur($couleur) {  
    $this->couleur = $couleur;  
}  
    //etc ...  
}
```



Création d'un objet

- En PHP5, la déclaration d'un objet revient à instancier une classe avec **new** :



//Déclaration d'une instance :

```
$MonTacot = new Motocyclette();
```

//Utilisation d'une instance

```
$MonTacot->setCouleur("rouge");
```

//Vérification

```
if($MonTacot instanceof Motocyclette)  
    echo "L'objet est du type Motocyclette";
```




Encapsulation



Les membres d'une classe PHP5 peuvent être **publics**, **protégés** ou **privés** :

```
class Demo {  
  
    //Attributs  
    private $aPrivate = "Attribut privé";  
    protected $aProtected = "Attribut protégé";  
    public $aPublic = "Attribut public";  
  
    //Méthodes  
    private function getPrivate() {  
        echo "get : ", $this->aPrivate, "<br />";  
    }  
    protected function getProtected() {  
        echo "get : ", $this->aProtected, "<br />";  
    }  
    public function getPublic() {  
        echo "get : ", $this->aPublic, "<br />";  
        $this->getProtected();  
        $this->getPrivate();  
    }  
}
```

```
$objet = new Demo();  
$objet->getPublic();  
$objet->getProtected(); //Erreur  
$objet->getPrivate(); //Erreur
```

3

Constructeur & Destructeur



Constructeur



Il faut ici distinguer les différentes versions :

- PHP 3 : une fonction portant le même nom que la classe
- PHP 4.x : une fonction membre portant le même nom que sa classe
- A partir de PHP5 : une fonction membre spécifique `__construct()`

Motocyclette
 <code>__couleur</code>
 <code>Motocyclette()</code>

```
class Motocyclette {  
    //attribut  
    private $couleur;  
  
    //Constructeur  
    function __construct($couleur="") {  
        echo "Constructeur<br />";  
        $this->couleur = $couleur;  
    }  
    //etc ...  
}
```

//Déclaration d'une instance

`$MonTacot1 = new Motocyclette();`

//ou :

`$MonTacot2 = new Motocyclette("rouge");`



Destructeur

- Il n'y a pas de destructeurs en PHP 4.x.
- Solution PHP 4.x : étant donné que le destructeur est appelé lorsqu'un objet est détruit, on peut alors simuler un destructeur avec `register_shutdown_function(string fct)`, qui permet d'enregistrer la fonction `fct` pour son exécution à la fin du script.
- PHP5 introduit la notion de destructeur avec la fonction membre spécifique `__destruct()`

```
function __destruct() {  
    echo "Destructeur<br />";  
}
```

- Destruction : on utilisera la fonction `unset($MonTacot)`.

4

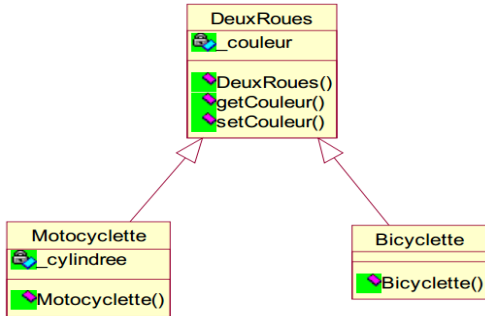
Héritage



Héritage en PHP



En PHP5, une classe peut être déclarée comme étant une sous-classe d'une autre classe en spécifiant le mot clé **extends** :



```
class DeuxRoues {

    private $_couleur;

    function __construct($couleur= "") {
        $this->_couleur = $couleur;
    }

    public function getCouleur() {
        return $this->_couleur;
    }
    //etc ...
}
```

```
class Bicyclette extends DeuxRoues {

    //Constructeur
    function __construct($couleur= "") {

        //Appel du constructeur parent
        parent::__construct($couleur);
    }
}
```

```
//Déclaration d'une instance :
$MonVelo = new Bicyclette("bleue");
echo $MonVelo->getCouleur();
```



Surcharge et Redéfinition

Surcharge (overloading)

- PHP ne permet pas la surcharge de fonction (ou de méthodes) et donc on ne peut attribuer le même nom à plusieurs fonctions. Par contre, la redéfinition est possible.

Redéfinition (overriding)

- PHP permet la redéfinition, c'est-à-dire la possibilité de redéclarer les mêmes attributs et opérations d'une super classe au sein d'une sous classe.

```
class DeuxRoues {  
    private $_couleur;  
    public function getCouleur() {  
        return $this->_couleur;  
    }  
    //etc ...  
}
```

```
class Bicyclette extends  
    DeuxRoues {  
    //redéfinition de la méthode  
    public function getCouleur(){  
        echo "Nouvelle méthode !";  
        return $this->_couleur;  
    }  
}
```

```
//Déclaration d'une instance :  
$MonVelo = new Bicyclette("bleue");  
echo $MonVelo->getCouleur();
```

Appel de cette méthode

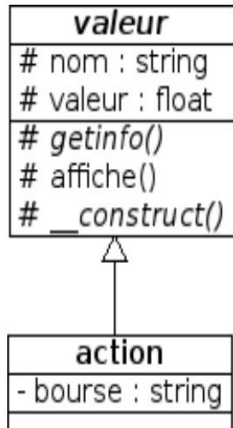
5

Abstract et Static



Classe abstraite

- Une classe abstraite est une classe qui ne peut pas être instanciée. Elle est utilisée pour être **héritée**.
- Une classe abstraite a au moins une **opération abstraite**. Mais elle peut aussi avoir des opérations concrètes.
- Remarque : une classe ne peut hériter que d'une seule classe, ce qui n'est pas le cas avec les **interfaces**.



```
abstract class valeur {
    protected $nom;
    protected $prix;

    abstract protected function getinfo();
    protected function affiche() {
        $info = "Le prix de $this->nom est de $this->prix";
        return $info;
    }
}
```

```
class action extends valeur {
    private $bourse;
}
```

```
$action1 = new action("Alcotel",9.76);
```



Résolution de portée

- Il est possible de faire référence aux méthodes d'une classe de base.
- Pour cela, on utilise l'opérateur :: en précisant soit le nom de la classe de base soit le mot clé self

```
class MaClasse {  
    public $attribut = "valeur";  
    public function Methode1() {  
        echo "Je suis une méthode publique !<br />";  
        $this->Methode2();  
        //Methode2(); Erreur : essaye d'appeler une fonction et non la méthode  
        MaClasse::Methode2();  
        self::Methode2();  
    }  
    private function Methode2() {  
        echo "Je suis une méthode privée !<br />";  
        echo "\$attribut = $this->attribut<br />";  
        //echo "MaClasse::attribut = ".MaClasse::$attribut; //Erreur : devrait être statique  
    }  
}
```

```
$objet = new MaClasse;  
$objet->Methode1();
```



Membre static



Les membres d'une classe PHP5 peuvent être statiques

```
class Bourse{  
    //Propriété statique  
    public static $lieu = "Paris";  
    //Méthodes statiques  
    public static function getheure(){  
        $heure = date("h : m : s");  
        return $heure;  
    }  
    public static function afficheInfo(){  
        $texte = Bourse::$lieu.", il est ".self::getheure();  
        return $texte;  
    }  
    public function setLieu($lieu){  
        Bourse::$lieu = $lieu;  
    }  
}
```

```
echo Bourse::$lieu;  
echo Bourse::getheure();  
echo Bourse::afficheInfo();
```

6

Les exceptions



Gestion des exceptions

- Une exception est un traitement qui sera appelé lorsqu'un cas particulier est détecté pendant le déroulement du programme.
- **Remarque** : une erreur est le résultat d'un fonctionnement anormal alors qu'une exception est un fonctionnement normal, mais exceptionnel.
- Les exceptions fonctionnent de la même façon dans tous les langages.
 - Dans un bloc défini (try = essayer), l'environnement d'exécution envoie un signal (throw = lancer)
 - Ce signal est envoyé au gestionnaire d'exception qui diffusera l'information aux différents blocs de traitement (catch = attraper).
 - On quitte le bloc try (sans exécuter le code restant du bloc try) et le bloc catch correspondant exécutera un code adapté.



Exemple

```
function verifier($var) {  
    if (empty($var)) {  
        throw new Exception ('La variable est vide !');  
    }  
    else {  
        return $var;  
    }  
}
```

```
try {  
    $var1 = "abc";  
    $var2 = "";  
    verifier($var1);  
    echo $var1."<br />";  
    verifier($var2);  
    echo $var2."<br />"; // code non exécuté  
}  
catch(Exception $myException) {  
    echo $myException->getMessage();  
}
```

7

Objet PDO



Présentation générale -1

- Le SGBD le plus connu pour accompagner le langage PHP est MySQL. Cependant, PHP peut aussi dialoguer avec plusieurs autres SGBD comme PostgreSQL, Oracle, SQL Server...
- Jusqu'à la version 5 de PHP, la connexion à une base de données (en particulier MySQL) se faisait d'une manière transparente grâce à des fonctions du genre `mysql_connect()`, `mysql_select_db()`, `mysql_query()`...
- Ces fonctions marchent encore sur les dernières versions PHP5.x (y compris PHP5.6), mais elles sont rendues obsolètes à partir de PHP5.5. A la sortie de PHP7 elles ont été supprimées.
- Bien que ces fonctions marchent très bien, leurs limites n'ont pas mis beaucoup de temps pour se manifester, en particulier quand on souhaite migrer vers un SGBD autre que MySQL.
- En effet, toutes ces fonctions préfixées par `mysql_` n'auraient pas marché. Il faut par conséquent repasser en vue une grande partie du code déjà développé.
- Heureusement l'objet PDO a été développé et intégré sous forme d'extension au langage PHP.



Présentation générale -2

- PDO (**PHP Data Objects**) est une interface qui permet au scripts PHP d'interroger une base de données via des requêtes SQL quelque soit le SGBD
- Autrement dit, si l'application Web repose sur le SGBD MySQL, on peut migrer vers le SGBD PostgreSQL sans modifier le code source (quelques modifications mineures sont requises).
- PDO est une extension qui s'ajoute au PHP pour que ses différentes fonctionnalités soient disponibles dans le langage.
- L'abstraction de la base de données constitue un point fort par rapport aux anciennes méthodes d'accès à celles-ci. D'ailleurs, il constitue l'ultime avantage du PDO, sans en être le seul.



Présentation générale -3

- Le bon fonctionnement de PDO repose sur la disponibilité du pilote de la base de données. Il faut que celui-ci soit pris en charge pour pouvoir interroger le SGBD souhaité.
- Pour déclarer le pilote du SGBD MySQL par exemple, il faut aller dans le fichier **php.ini** et ajouter la ligne suivante (si elle n'est pas déjà déclarée)

```
extension = php_pdo_mysql.dll
```

- Notez que la ligne suivante doit aussi figurer:

```
extension = php_pdo.dll
```



Connexion à une BD

- La **Chaîne de connexion** est la représentation textuelle des informations de connexion à la base de données installée sur un serveur.
- La déclaration de la chaîne de connexion dans ce cas revient à créer une instance PDO comme ceci:

```
$pdo = new PDO("mysql:host=server;dbname=db","user","pswd");
```

- Le mot clé mysql s'appelle **préfixe DSN** (pour **Data Source Name**). C'est pratiquement le seul paramètre qui change quand on passe d'un SGBD à un autre (d'où le nom abstraction de la base de données).



Gestion des exceptions

- Si des paramètres de connexion ne sont pas bien déclarés ou que le serveur de base de données n'est pas accessible alors une exception de type **PDOException** est lancée automatiquement.
- Il faut donc la rattraper comme ceci:

```
try{  
    $pdo=new PDO("mysql:host=localhost;dbname=mabase","user","1234");  
}  
catch(PDOException $e){  
    echo $e->getMessage();  
}
```



Requêtes préparées -1

- L'un des points fort de PDO est les **requêtes préparées**. En effet, une requête préparée est plus rapide et, aussi, plus sûre (surtout contre les tentatives d'injections SQL).
- Pour préparer une requête, il est recommandé de ne pas y spécifier les valeurs, mais plutôt remplacer celle-ci par des **marqueurs interrogatifs (?)** ou des **paramètres nommés**.

```
<?php
    $ins = $pdo->prepare("insert into utilisateurs (nom,prenom,login,pass)values(?,?,?,?)");
    $ins->execute(array("ahmed","kacimi","a.kacimi", md5("2026")));

//ou bien
    $ins = $pdo->prepare("insert into utilisateurs (nom,prenom,login,pass)
values(:nom,:prenom,:login,:pass)");
    $ins->execute(array(":prenom"=>"ahmed", ":pass"=>md5("2026"), ":nom"=>"kacimi",
":login"=>"a.kacimi"));
?>
```



Requêtes préparées -2

- De la même manière on peut passer des requêtes de modification, suppression, vidage de la table...
- Exemple de suppression de l'enregistrement inséré:

```
<?php
    $ins = $pdo->prepare("delete from utilisateurs where id=? limit 1");
    $ins->execute(array(1));
?>
```



Requête de sélection



Supposons maintenant que nous souhaitons afficher des lignes d'une requête de sélection dans une page Web. On va tout d'abord préparer puis exécuter la requête de sélection:

```
$ins = $pdo->prepare("select * from utilisateurs order by id");  
$ins->setFetchMode(PDO::FETCH_ASSOC);  
$ins->execute();  
$tab = $ins->fetchAll();
```



on va récupérer toute la table dans une seule variable. Alors, il est évident que la variable sera un tableau à deux dimensions. Une pour désigner la ligne et l'autre pour désigner la colonne.



La méthode `fetchAll()` appelée par l'objet requête retourne un tableau contenant toutes les lignes retournées par la requête.



Modes de récupération

Avec PDO, il existe plusieurs modes de récupération (**fetch modes**) en plus de PDO::FETCH_ASSOC. Voici les principaux :

- PDO::FETCH_ASSOC → Retourne un tableau associatif (clé = nom des colonnes)
- PDO::FETCH_NUM → Retourne un tableau indexé numériquement
- PDO::FETCH_BOTH (par défaut) → Combine associatif + numérique
- PDO::FETCH_OBJ → Retourne un objet
- PDO::FETCH_CLASS → Retourne un objet d'une classe personnalisée
- PDO::FETCH_COLUMN → Retourne une seule colonne